1. Aim of the Experiment

To implement and train the same Multilayer Perceptron (MLP) using Sigmoid, Tanh and ReLU activation functions, compare their training speed and classification accuracy, visualize their gradient flow, and study the vanishing and exploding gradient problems.

2. Theory

Activation functions introduce non-linearity into neural networks, allowing them to learn complex relationships between inputs and outputs. The choice of activation function affects gradient propagation, convergence speed and model performance.

A. Sigmoid Activation Function

The Sigmoid function maps input values to a range between 0 and 1: $\sigma(x) = 1 / (1 + e^{-x})$.

- Produces smooth outputs between 0 and 1.
- Commonly used for binary classification output layers.
- Its gradient becomes very small for large positive or negative inputs, potentially causing vanishing gradients in deep networks.

B. Tanh Activation Function

The Tanh function maps input values to a range between −1 and +1: $\tanh(x) = (e^{x} - e^{-x}) / (e^{x} + e^{-x})$.

- Produces zero-centered outputs.
- Often provides stronger gradients near zero than Sigmoid.
- Can still suffer from vanishing gradients when inputs are saturated at large magnitudes.

C. ReLU Activation Function

The Rectified Linear Unit (ReLU) function is defined as: $\text{ReLU}(x) = \max(0, x)$.

- Outputs zero for negative inputs and the input itself for positive inputs.
- Is computationally simple and often supports efficient training.
- Can experience the dying ReLU problem when neurons consistently receive negative inputs and stop updating.

3. Dataset Used

The dataset depends on the implementation. MNIST is a suitable example if the experiment uses handwritten digit classification.

Property	MNIST (if used)
Total images	70,000
Training images	60,000
Testing images	10,000
Image dimensions	28 × 28 grayscale

Number of classes	10 (digits 0–9)
Flattened input features	784
Preprocessing	Normalize pixel values to 0–1

Use the same training and testing data for all three models to ensure a fair comparison. If a different dataset is used in the code, replace this section with its actual details.

4. Architecture of the Models

Three MLP models are constructed with identical layer sizes, weight initialization strategy, optimizer, loss function, batch size and training data. Only the activation function in the hidden layers changes.

Layer	Configuration	Activation comparison
Input	784 features (MNIST)	Same for all
Hidden Dense 1	128 neurons	Sigmoid / Tanh / ReLU
Hidden Dense 2	64 neurons	Sigmoid / Tanh / ReLU
Output	10 neurons	Softmax for all

Common Training Configuration

Parameter	Configuration
Model	MLP
Hidden layers	2 **(This may change according to your implementation)**
Hidden neurons	128, 64 **(This may change according to your implementation)**
Optimizer	Same for all models
Loss function	Sparse categorical cross-entropy
Metric	Accuracy
Epochs	Same for all models
Batch size	Same for all models

5. Algorithm

1. Start the experiment and import TensorFlow/Keras, NumPy and Matplotlib.
2. Load the dataset and separate training and testing data.
3. Normalize the input images and flatten them into feature vectors.
4. Define a common MLP architecture with two hidden layers and an output layer.
5. Create three models using Sigmoid, Tanh and ReLU in the hidden layers.
6. Compile all three models using the same optimizer, loss function and evaluation metric.
7. Train each model using identical training conditions.
8. Record training time, training accuracy, validation accuracy and test accuracy.
9. Plot training and validation accuracy and loss curves.
10. Calculate or record gradient magnitudes at selected layers during backpropagation.
11. Plot and compare gradient flow for all three activation functions.
12. Analyze convergence speed, accuracy and gradient behavior.
13. Discuss vanishing gradients, exploding gradients and the dying ReLU problem.
14. End the experiment.

6. Flowchart

START → Import libraries → Load dataset → Normalize and preprocess → Define common MLP → Create Sigmoid, Tanh and ReLU models → Compile with same settings → Train all models → Record time and

accuracy → Measure and visualize gradients → Plot learning curves → Compare results → Discuss gradient issues → END

7. Analysis and Discussion

A. Comparison of training speed

Training speed can be measured by total training time and the number of epochs required to reach a selected accuracy. ReLU often converges efficiently because its derivative is 1 for positive inputs, avoiding saturation on that side. Sigmoid and Tanh may converge more slowly when many neurons operate in saturated regions.

B. Comparison of accuracy

All three models are evaluated on the same test set. Differences in accuracy can arise from how each activation function transforms inputs and propagates gradients. Accuracy should be examined together with loss curves and gradient measurements.

C. Gradient flow

During backpropagation, gradients are multiplied by activation derivatives and layer weights. If these products become very small, early layers receive little learning signal. If they become excessively large, training can become unstable.

D. Vanishing and exploding gradients

Sigmoid derivatives are at most 0.25 and approach zero in saturated regions. Tanh has its strongest derivative near zero but also saturates. ReLU has derivative 1 for positive inputs and 0 for negative inputs, which can preserve gradients through active units but may leave inactive units without updates. Exploding gradients can also arise from weight initialization, network depth and optimization settings, not just the activation function.

E. Remedies and interpretation

Suitable weight initialization, careful learning-rate selection, normalization and gradient clipping can help manage gradient problems. The measured plots and results should be used to determine how these issues appear in the experiment.

8. Observation Table

Fill in the measured results after executing the experiment. Do not estimate or assume numerical outcomes.

Sr. No.	Parameter	Sigmoid	Tanh / ReLU
1	Training time (seconds)	_____	Tanh: _____ ; ReLU: _____
2	Training accuracy (%)	_____	Tanh: _____ ; ReLU: _____
3	Validation accuracy (%)	_____	Tanh: _____ ; ReLU: _____
4	Test accuracy (%)	_____	Tanh: _____ ; ReLU: _____
5	Final training loss	_____	Tanh: _____ ; ReLU: _____
6	Final validation loss	_____	Tanh: _____ ; ReLU: _____

Gradient-flow observations

Observation	Sigmoid	Tanh	ReLU
Gradient magnitude in early layers	_____	_____	_____
Gradient magnitude in later layers	_____	_____	_____

Evidence of vanishing gradients	_____	_____	_____
Evidence of inactive neurons	_____	_____	_____
Training stability	_____	_____	_____

Suggested plots: (1) training and validation accuracy versus epoch, (2) training and validation loss versus epoch, (3) training time comparison, and (4) gradient magnitude by layer, preferably on a logarithmic scale when magnitudes vary widely.

9. Conclusion

In this experiment, three MLP models using Sigmoid, Tanh and ReLU activation functions are trained and compared under the same experimental conditions. Their training speed, classification accuracy and gradient flow are examined to understand how activation functions affect neural network learning.

Sigmoid and Tanh can suffer from vanishing gradients when their activations saturate, while ReLU can maintain gradients for positive inputs but may produce inactive neurons. Exploding gradients can also occur due to factors such as weight initialization and unstable gradient propagation. The final comparison should be based on the recorded accuracy, training time, loss curves and measured gradients rather than assuming one activation function performs best in every setting.